

SkillBridge AI

Master Product Requirements & Implementation Specification

Document Version: 1.0

Project Type: AI-powered career intelligence and skill-gap analysis platform

Target: Portfolio / Python Backend Developer placement project

Primary Users: Students, fresh graduates, job seekers, early-career professionals

Backend: Python + Django + Django REST Framework

Database: MySQL

Frontend: HTML + CSS + Vanilla JavaScript + Bootstrap 5

AI: Google Gemini API

Document Processing: pdfplumber / python-docx where required

Charts: Chart.js

Authentication: Django Authentication + JWT for REST APIs

Version Control: Git + GitHub

Deployment Target: Render

1. PRODUCT VISION

Build **SkillBridge AI**, an AI-powered career intelligence platform that helps candidates understand the gap between:

What they currently know

and

What the job market requires for the role they want.

The platform should not simply provide an ATS score.

It should answer:

"If I want this job, where do I currently stand, what am I missing, what should I improve first, and what should I do next?"

SkillBridge AI analyzes a candidate's resume against a target Job Description (JD), identifies skill gaps, prioritizes those gaps, recommends an actionable learning roadmap, and provides interview preparation based on the identified gaps.

2. REAL-WORLD PROBLEM

Many students and job seekers apply to jobs without understanding their actual fit.

A typical candidate may:

- read a Job Description
- upload a resume
- receive an ATS score
- see a list of missing keywords
- have no idea what to learn next

This creates a major gap between:

Candidate → Job Requirements

SkillBridge AI aims to bridge this gap.

3. CORE PRODUCT IDEA

The main workflow is:

Resume

+

Job Description

↓

AI Career Analysis

↓

Current Skills

↓

Required Skills

↓

Skill Gap

↓

Priority Analysis

↓

Personalized Roadmap

↓

Interview Preparation



Progress Tracking

4. PRODUCT OBJECTIVES

The application must allow users to:

1. create an account
 2. create a career profile
 3. upload a resume
 4. paste or upload a Job Description
 5. analyze resume vs Job Description
 6. identify matched skills
 7. identify missing skills
 8. identify partially matched skills
 9. calculate a transparent job-readiness score
 10. prioritize skill gaps
 11. generate a personalized learning roadmap
 12. recommend projects/practice activities
 13. generate interview preparation topics
 14. save previous analyses
 15. compare analyses
 16. track learning progress
 17. view career analytics
-

5. TARGET USERS

5.1 Student

A student preparing for campus placements.

Typical goal:

"I want to become a Python Backend Developer."

The student wants to know what skills are missing.

5.2 Fresher

A recent graduate applying for entry-level positions.

5.3 Early-Career Professional

Someone changing roles or moving into a new technical specialization.

6. MVP PRINCIPLE

The MVP should focus on one extremely strong workflow:

Resume + Job Description → Career Gap Analysis → Personalized Roadmap

Do not attempt to become LinkedIn, Coursera, Indeed, or a complete recruitment platform.

7. PRIMARY USER JOURNEY

Register

↓

Create Career Profile

↓

Upload Resume

↓

Paste Job Description

↓

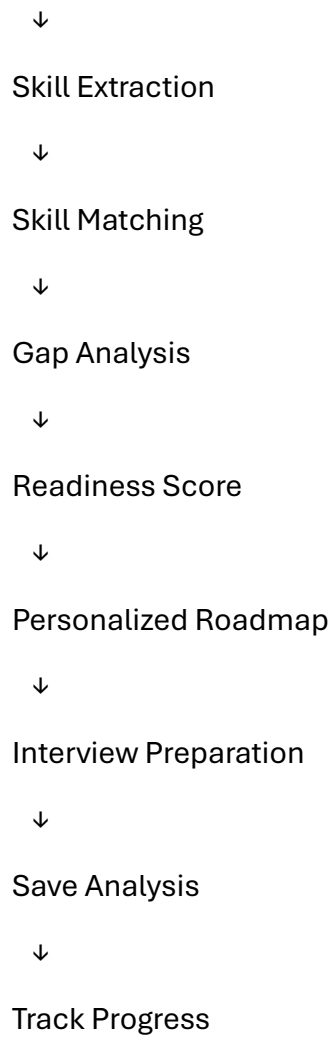
Start Analysis

↓

Resume Parsing

↓

Job Description Parsing



8. FEATURE 1 — AUTHENTICATION

Implement:

- registration
- login
- logout
- password hashing
- protected dashboard
- JWT authentication for REST APIs

Use Django's built-in authentication system.

Use Django REST Framework Simple JWT for API authentication.

Users must only access their own career data.

9. FEATURE 2 — CAREER PROFILE

After registration, users can create a profile.

Fields:

- full name
- target role
- experience level
- education
- current skills
- preferred technologies
- years of experience
- career goal
- weekly learning hours

Example:

Target Role:

Python Backend Developer

Experience:

Fresher

Current Skills:

Python

Django

MySQL

HTML

CSS

JavaScript

Weekly Learning Time:

15 hours

10. TARGET ROLE

Allow users to specify a target role.

Examples:

- Python Backend Developer
- Django Developer
- Java Developer
- Data Analyst
- AI/ML Engineer
- Frontend Developer
- Full Stack Developer

The MVP should NOT require role-specific machine-learning models.

AI should interpret the role based on the provided Job Description.

11. FEATURE 3 — RESUME UPLOAD

Allow:

- PDF resume
- DOCX resume

For MVP:

.pdf

.docx

Do not support arbitrary formats.

Validate:

- extension
- file type
- file size

12. RESUME PARSING

Extract useful information from the resume.

Extract:

Personal Information

- name
- email
- phone where present

Education

- degree
- institution
- graduation year

Experience

- company
- role
- duration
- responsibilities

Projects

- project names
- descriptions
- technologies

Skills

- programming languages
- frameworks
- databases
- tools
- cloud technologies
- other technical skills

Certifications

Where identifiable.

13. RESUME PARSING PRINCIPLE

Do not depend entirely on Gemini for basic document extraction.

Use deterministic extraction first where practical.

Conceptual workflow:

Resume

↓

Text Extraction

↓

Section Detection

↓

AI Structured Extraction

↓

Normalized Candidate Profile

Gemini can help identify and structure information that deterministic parsing cannot reliably categorize.

14. FEATURE 4 — JOB DESCRIPTION INPUT

Users can provide a Job Description in two ways.

Method A — Paste

Large text area.

Method B — Upload

Allow:

- PDF
- DOCX

The system extracts the text.

15. JOB DESCRIPTION PARSING

Extract:

- company if available
- role title
- experience requirement
- required skills
- preferred skills
- programming languages
- frameworks
- databases
- tools
- soft skills
- education requirements
- responsibilities
- qualifications

Example:

Role:

Python Backend Developer

Required:

Python

Django

REST APIs

SQL

Git

Preferred:

Docker

AWS

Redis

Experience:

0–2 years

16. FEATURE 5 — SKILL EXTRACTION

Normalize skills before matching.

For example:

Python 3

Python

Python Programming

should be recognized as:

Python

Likewise:

Django REST Framework

DRF

Django REST

can be normalized as:

Django REST Framework

Create a normalized internal skill representation.

17. SKILL CATEGORIES

Use categories such as:

Programming Languages

- Python
- Java

- JavaScript
- C++
- etc.

Frameworks

- Django
- Flask
- Spring
- React
- Angular
- etc.

Databases

- MySQL
- PostgreSQL
- MongoDB
- Redis
- etc.

Tools

- Git
- Docker
- Postman
- Jenkins
- etc.

Cloud

- AWS
- Azure
- GCP

Concepts

- REST APIs

- OOP
- DSA
- SQL
- System Design
- Testing

Soft Skills

- Communication
- Teamwork
- Problem Solving

18. FEATURE 6 — SKILL MATCHING

Compare:

Candidate Skills

vs

Job Requirements

Classify each skill as:

Matched

Candidate clearly demonstrates the skill.

Partial

Candidate mentions related knowledge or experience but does not strongly demonstrate the requirement.

Missing

Skill is required but not demonstrated.

Optional

Skill is preferred rather than required.

19. SKILL MATCH EXAMPLE

Job Description:

Python

Django

REST API

Docker

AWS

MySQL

Git

Candidate:

Python

Django

MySQL

Git

Result:

MATCHED

✓ Python

✓ Django

✓ MySQL

✓ Git

MISSING

X Docker

X AWS

PARTIAL

~ REST APIs

20. FEATURE 7 — JOB READINESS SCORE

Generate a score from 0–100.

The score must be transparent.

Do NOT allow Gemini to arbitrarily generate the final score.

The backend should calculate the score using a documented algorithm.

Example conceptual weighting:

Required Skills 60%

Experience/Projects 20%

Technical Evidence 10%

Preferred Skills 10%

The exact formula should be implemented as a dedicated scoring service and documented.

21. SCORE EXAMPLE

Job Readiness

78 / 100

Strong Areas

✓ Python

✓ Django

✓ SQL

✓ Git

Needs Improvement

⚠ REST APIs

⚠ Docker

⚠ Testing

Major Gap

22. SCORE INTERPRETATION

Use:

90–100 → Highly Ready

75–89 → Strong Candidate

60–74 → Developing

40–59 → Significant Gaps

0–39 → Early Stage

These labels are guidance, not guarantees of employment.

The UI must explicitly avoid claiming:

"You will definitely get this job."

23. FEATURE 8 — SKILL GAP ANALYSIS

For every missing or partial skill, calculate:

- importance
- current status
- priority
- reason
- recommended action

Example:

Docker

Status:

Missing

Importance:

High

Why:

Listed as a preferred technology and frequently used in backend deployment workflows.

Priority:

3

Recommendation:

Learn Docker fundamentals and containerize one Django application.

24. SKILL PRIORITY

Prioritize skills based on:

1. Required vs preferred
2. Frequency in JD
3. relevance to target role
4. relationship to other missing skills
5. candidate's current skill level

Priority levels:

Critical

High

Medium

Low

25. FEATURE 9 — PERSONALIZED LEARNING ROADMAP

This is one of the most important features.

Do not simply generate:

Learn Docker

Learn AWS

Learn Redis

Instead generate a sequence.

Example:

Week 1

REST API Fundamentals

Week 2

Django REST Framework

Week 3

Docker Fundamentals

Week 4

Deploy Django with Docker

Week 5

AWS Fundamentals

The roadmap should account for:

- current skill level
- target role
- missing skills
- weekly available hours
- dependencies between skills

26. ROADMAP STRUCTURE

Each roadmap item should contain:

- title
- skill
- priority
- estimated hours
- prerequisites
- learning objectives
- recommended practice
- project exercise
- completion status

Example:

Skill:

Docker

Estimated Time:

8 hours

Prerequisite:

Basic Linux commands

Objectives:

- Understand images
- Understand containers
- Write Dockerfile
- Containerize Django

Practice:

Containerize your Django project.

Status:

Not Started

27. FEATURE 10 — PROJECT RECOMMENDATIONS

For important skill gaps, recommend practical projects.

Example:

Missing:

REST APIs

Docker

Redis

Recommendation:

Build a Django REST API and containerize it using Docker.

The goal is to make the learning roadmap practical.

28. FEATURE 11 — INTERVIEW PREPARATION

Generate interview preparation based on:

- target role
- Job Description
- missing skills
- resume projects
- candidate experience

Generate categories:

Technical Questions

Example:

Explain Django ORM and how it differs from writing raw SQL.

Project Questions

How did you use Django in your project?

Skill-Gap Questions

If Docker is missing:

What problem does Docker solve?

HR/Behavioral Questions

Why are you interested in this role?

29. INTERVIEW QUESTION DIFFICULTY

Use:

- Beginner
- Intermediate
- Advanced

For a fresher, default to:

60% Beginner

30% Intermediate

10% Advanced

The exact distribution can be adjusted based on the role.

30. FEATURE 12 — RESUME IMPROVEMENT

After analysis, provide suggestions such as:

- missing measurable achievements
- skills not clearly demonstrated
- projects that could be explained better
- weak bullet points
- missing technologies relevant to the JD

Do not fabricate experience.

Never tell the user to claim a skill they do not actually possess.

Recommendations must be based on the user's actual resume.

31. FEATURE 13 — ANALYSIS HISTORY

Every completed analysis should be saved.

History should show:

- target role
- Job Description title
- readiness score
- matched skills
- missing skills
- date
- roadmap generated

Users can open previous analyses.

32. FEATURE 14 — PROGRESS TRACKING

Users can mark roadmap items:

Not Started

In Progress

Completed

Track:

- completed skills
 - completed learning tasks
 - completed projects
 - learning hours
-

33. RE-ANALYSIS

A user should be able to return later and analyze the same Job Description again after improving their skills.

Example:

First analysis

Readiness: 62

After learning Docker and REST APIs:

Second analysis

Readiness: 81

The system should show:

+19 improvement

This turns SkillBridge from a one-time analyzer into a career progression tool.

34. DASHBOARD

Create a professional dashboard.

Display:

Career Goal

Python Backend Developer

Current Readiness

78 / 100

Top Skills

Python

Django

SQL

Git

Critical Gaps

Docker

REST APIs

Testing

Roadmap Progress

12 / 20 tasks completed

Recent Analyses

Show previous Job Description analyses.

Skill Progress Chart

Use Chart.js.

35. ANALYTICS

Dashboard charts should include:

Readiness Trend

Show readiness scores across analyses.

Skill Coverage

Matched vs partial vs missing.

Roadmap Progress

Completed vs remaining.

Skill Category Distribution

Programming / Framework / Database / Tools / Cloud etc.

Keep analytics simple and meaningful.

36. FRONTEND

Use:

- HTML5
- CSS3
- Vanilla JavaScript
- Bootstrap 5
- Chart.js

Do NOT use React.

Do NOT introduce another frontend framework.

The interface should be:

- modern
- professional
- responsive
- clean

- easy to understand

Recommended design:

- dark/light professional dashboard
- clear readiness score
- skill badges
- progress bars
- roadmap cards
- clean charts

37. REQUIRED PAGES

Public Pages

/

/about

/login

/register

Authenticated Pages

/dashboard

/profile

/resume

/analyze

/analysis/<id>

/roadmap

/interview-prep

/history

/settings

Exact URLs may be adjusted during implementation.

38. ANALYSIS PAGE

The main analysis form should contain:

New Career Analysis

Target Role

[Python Backend Developer]

Resume

[Upload PDF/DOCX]

Job Description

[Paste Job Description]

Weekly Learning Hours

[10]

Current Experience Level

[Fresher]

[Analyze My Career Fit]

Display a loading state during analysis.

39. ANALYSIS RESULT PAGE

The most important page.

Example:

Python Backend Developer

Job Readiness

78 / 100

Strong Candidate

SKILL MATCH

Matched 7

Partial 2

Missing 3

Matched Skills

✓ Python

✓ Django

✓ MySQL

✓ Git

Skill Gaps

HIGH

△ REST APIs

HIGH

△ Docker

MEDIUM

△ Automated Testing

Recommended Roadmap

1. REST APIs
2. Django REST Framework
3. Docker
4. Testing

[View Full Roadmap]

[Prepare For Interview]

40. DATABASE DESIGN

Use MySQL.

Minimum models:

User

Use Django authentication.

CareerProfile

Fields:

- id
 - user
 - target_role
 - experience_level
 - education
 - current_skills
 - weekly_learning_hours
 - career_goal
 - created_at
 - updated_at
-

Resume

Fields:

- id
 - user
 - file
 - extracted_text
 - parsed_data
 - uploaded_at
-

JobDescription

Fields:

- id
 - user
 - title
 - company
 - source_text
 - extracted_text
 - parsed_data
 - created_at
-

Analysis

Fields:

- id
 - user
 - resume
 - job_description
 - readiness_score
 - summary
 - created_at
-

Skill

Fields:

- id
 - name
 - normalized_name
 - category
-

AnalysisSkill

Fields:

- id
- analysis
- skill
- status
- importance
- priority
- evidence
- recommendation

Status:

matched

partial

missing

optional

Roadmap

Fields:

- id
- analysis
- title
- description
- total_estimated_hours
- created_at

RoadmapItem

Fields:

- id
- roadmap
- skill

- title
 - description
 - estimated_hours
 - priority
 - prerequisites
 - learning_objectives
 - practice_task
 - status
 - order
-

InterviewQuestion

Fields:

- id
 - analysis
 - category
 - difficulty
 - question
 - expected_topics
 - created_at
-

ProgressRecord

Fields:

- id
- user
- roadmap_item
- status
- completed_at
- notes

41. DATABASE RELATIONSHIP

Conceptually:

User

|

|— CareerProfile

|

|— Resume

|

|— JobDescription

|

|— Analysis

| |

| |— AnalysisSkill

| |

| |— Roadmap

| | |

| | |— RoadmapItem

| |

| |— InterviewQuestion

|

|— ProgressRecord

42. SKILL MASTER DATABASE

Create a reusable Skill table.

Example initial skills:

Python

Django

Django REST Framework

Flask

FastAPI

Java

Spring Boot

JavaScript

React

Angular

HTML

CSS

MySQL

PostgreSQL

MongoDB

Redis

Git

Docker

AWS

Azure

GCP

REST APIs

GraphQL

SQL

OOP

DSA

Testing

Linux

CI/CD

This list can be expanded later.

AI should be able to discover skills not present in the initial database, but normalize them before storing where practical.

43. REST API

Use Django REST Framework.

Authentication

POST /api/auth/register/

POST /api/auth/login/

POST /api/auth/refresh/

Profile

GET /api/profile/

PUT /api/profile/

Resume

POST /api/resumes/

GET /api/resumes/

GET /api/resumes/<id>/

DELETE /api/resumes/<id>/

Job Descriptions

POST /api/jobs/

GET /api/jobs/

GET /api/jobs/<id>/

DELETE /api/jobs/<id>/

Analysis

POST /api/analyses/

GET /api/analyses/

GET /api/analyses/<id>/

Roadmap

GET /api/analyses/<id>/roadmap/

PATCH /api/roadmap/items/<id>/

Interview

GET /api/analyses/<id>/interview-questions/

Dashboard

GET /api/dashboard/

Exact endpoint structure may be refined during implementation.

44. API REQUIREMENTS

Use appropriate HTTP status codes:

200 OK

201 Created

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

500 Internal Server Error

Validate all input.

Never trust frontend validation.

Protect all user-specific endpoints.

45. AI ARCHITECTURE

Do not put Gemini calls directly into Django views.

Create a dedicated AI service.

Conceptually:

Django API

↓

Career Analysis Service

↓

Resume Parser



JD Parser



Skill Normalizer



Skill Matcher



Gemini AI



Structured Response Validator



Scoring Engine



Roadmap Generator



Database

Keep responsibilities separate.

46. AI RESPONSIBILITIES

Gemini should be used for:

- structured resume interpretation
- Job Description interpretation
- skill extraction
- semantic skill matching
- explanation of skill gaps
- roadmap generation
- project recommendations

- interview question generation
- resume improvement suggestions

The backend should handle:

- authentication
- authorization
- database operations
- file validation
- scoring
- progress tracking
- data relationships
- API behavior

Do not let AI control critical application logic.

47. STRUCTURED AI RESPONSE

AI responses should preferably be structured JSON.

Example:

```
{
  "role": "Python Backend Developer",
  "summary": "The candidate has a strong foundation...",
  "matched_skills": [
    {
      "skill": "Python",
      "evidence": "Python projects and skills section"
    }
  ],
  "partial_skills": [
    {
      "skill": "REST APIs",
```

```
      "reason": "Mentioned indirectly but lacks strong project evidence"
    }
  ],
  "missing_skills": [
    {
      "skill": "Docker",
      "importance": "high",
      "reason": "Preferred requirement in the job description"
    }
  ]
}
```

Validate AI responses before storing them.

48. AI FAILURE HANDLING

If Gemini fails:

- do not crash the entire application
- show an understandable error
- preserve uploaded resume and Job Description
- allow retry
- log technical error
- avoid exposing API details to the user

Where practical, deterministic skill matching should still operate.

49. AI HALLUCINATION CONTROL

The system must never invent candidate experience.

For example, if the resume does not contain Docker:

Do NOT generate:

"The candidate has experience with Docker."

Instead:

"Docker was not found in the provided resume."

AI-generated recommendations must distinguish between:

Evidence found

and

Recommendation

50. NO GUARANTEE REQUIREMENT

The platform must never claim:

- "You will get this job."
- "You are guaranteed to be selected."
- "This score guarantees employment."

Use language such as:

"Estimated job-readiness based on the provided resume and Job Description."

51. SCORING ENGINE

Create a dedicated backend scoring service.

Example conceptual algorithm:

Required Skill Coverage

+

Preferred Skill Coverage

+

Evidence Quality

+

Experience Alignment

+

Project Alignment

=

Readiness Score

The exact weights must be documented.

Example:

Required skills 50%

Preferred skills 15%

Experience alignment 15%

Project evidence 10%

Skill evidence 10%

Do not use an opaque AI-generated number as the final score.

52. SKILL MATCHING ENGINE

The matching process should have multiple layers.

Layer 1 — Exact matching

Python → Python

Layer 2 — Alias matching

DRF → Django REST Framework

Layer 3 — Semantic AI matching

Use Gemini when two skills may be related but not identical.

Example:

REST API

Django REST Framework

The system can determine that DRF is evidence of REST API knowledge.

Do not assume every related technology is equivalent.

53. RESUME DATA PRIVACY

Treat resumes as private user data.

Users must only access their own resumes.

Do not expose resumes publicly.

Do not put resume contents into logs.

Do not expose extracted resume text through unauthorized API endpoints.

54. FILE SECURITY

Only allow:

.pdf

.docx

Set a reasonable upload limit.

Reject executable files.

Validate files server-side.

Store uploaded files securely.

55. ENVIRONMENT VARIABLES

Use .env.

Example:

SECRET_KEY=your-secret-key

DEBUG=True

GEMINI_API_KEY=your-gemini-api-key

DB_NAME=skillbridge

DB_USER=root

DB_PASSWORD=your-password

DB_HOST=localhost

DB_PORT=3306

Never hardcode secrets.

Never commit .env.

Create:

.env.example

56. ERROR HANDLING

Handle:

- invalid resume
- corrupted PDF
- corrupted DOCX
- empty Job Description
- invalid file type
- oversized upload
- AI API failure
- malformed AI response
- database errors
- unauthorized access
- missing analysis
- invalid roadmap item
- network errors

Use user-friendly messages.

Do not expose stack traces.

57. TESTING

Implement tests for:

Authentication

- registration
- login
- protected routes

Profile

- create

- update
- authorization

Resume

- valid PDF
- valid DOCX
- invalid file
- oversized file

Job Description

- create
- validation
- authorization

Analysis

- score calculation
- skill matching
- analysis creation

Roadmap

- generation
- progress update

Security

- user A cannot access user B's analysis
- unauthorized API access is rejected

Mock Gemini API calls during automated testing.

Do not make tests dependent on a live AI API.

58. LOGGING

Use Python/Django logging.

Log:

- analysis started

- analysis completed
- parsing failures
- AI failures
- unexpected backend errors

Do NOT log:

- passwords
- Gemini API keys
- complete resumes
- unnecessary personal information

59. PERFORMANCE

For MVP:

- paginate analysis history
- avoid unnecessary database queries
- use `select_related` / `prefetch_related`
- limit resume size
- limit excessive AI requests
- avoid sending unnecessary text to Gemini

Do not introduce Redis, Celery, Kafka, Elasticsearch, or microservices unless the implementation genuinely requires them.

60. DASHBOARD ANALYTICS

Dashboard should calculate:

Total Analyses

Average Readiness

Highest Readiness

Skills Improved

Roadmap Completion

Charts:

Readiness Over Time

Line chart.

Skill Coverage

Matched / Partial / Missing.

Roadmap Progress

Completed / Remaining.

61. ANALYSIS COMPARISON

Allow users to compare two analyses.

Example:

Previous

Readiness: 62

Missing:

Docker

REST APIs

Testing

Current

Readiness: 81

Missing:

Testing

Improvement:

+19 points

This demonstrates the value of the platform.

62. INTERVIEW PREPARATION PAGE

Display:

Target Role:

Python Backend Developer

Based on your gaps:

1. Django REST Framework
2. Docker
3. API Design

Generate questions such as:

Beginner:

What is a REST API?

Intermediate:

How does authentication work in DRF?

Project:

How would you design a Django API for this system?

Include:

- category
- difficulty

- question
- expected concepts

Do not necessarily generate full answers in the MVP.

63. ROADMAP PAGE

Example:

Your Career Roadmap

Goal:

Python Backend Developer

1. REST API Fundamentals

Priority: Critical

Estimated: 6 hours

Status: Completed

2. Django REST Framework

Priority: High

Estimated: 10 hours

Status: In Progress

3. Docker

Priority: High

Estimated: 8 hours

Status: Not Started

4. Automated Testing

Priority: Medium

Estimated: 6 hours

Status: Not Started

64. PRODUCT RECOMMENDATION ENGINE

Recommendations should be practical.

Bad:

Learn Docker.

Good:

Learn Docker fundamentals, then containerize a Django REST API and run the application with a MySQL database.

Every major skill recommendation should ideally include:

- what to learn
 - why it matters
 - estimated effort
 - practical exercise
 - expected outcome
-

65. FRONTEND STATES

Every important page must have:

Loading state

Example:

Analyzing your resume and Job Description...

Empty state

Example:

You haven't completed an analysis yet.

Error state

Example:

We couldn't analyze this resume. Please check the file and try again.

Success state

Show results clearly.

66. LANDING PAGE

Landing page should communicate the problem immediately.

Hero

SkillBridge AI

Understand the gap between your skills and your next career opportunity.

CTA:

Analyze My Career

Secondary CTA:

See How It Works

67. LANDING PAGE SECTIONS

Include:

1. Problem
2. How SkillBridge works
3. Resume + JD analysis
4. Skill gap detection
5. Personalized roadmap

6. Interview preparation
7. Progress tracking
8. Technology
9. Privacy
10. CTA

Avoid unsupported claims.

68. BACKEND ARCHITECTURE

Recommended Django apps:

accounts/

profiles/

resumes/

jobs/

analysis/

roadmap/

interviews/

dashboard/

ai_engine/

Responsibilities:

accounts

Authentication.

profiles

Career profile.

resumes

Resume upload and parsing.

jobs

Job Description management.

analysis

Skill matching and scoring.

roadmap

Learning roadmap.

interviews

Interview preparation.

dashboard

Analytics.

ai_engine

Gemini integration.

69. RECOMMENDED PROJECT STRUCTURE

skillbridge-ai/

|

├─ manage.py

|

├─ config/

|

├─ settings.py

|

├─ urls.py

|

├─ asgi.py

|

└─ wsgi.py

|

├─ accounts/

├─ profiles/

├─ resumes/

├─ jobs/

├─ analysis/

└─ roadmap/

```
├── interviews/
├── dashboard/
├── ai_engine/
|
├── templates/
├── static/
├── media/
|
├── tests/
|
├── requirements.txt
├── .env.example
├── .gitignore
└── README.md
```

The exact structure can be adjusted if required, but maintain separation of concerns.

70. REST API STRUCTURE

Recommended:

```
/api/auth/
/api/profile/
/api/resumes/
/api/jobs/
/api/analyses/
/api/roadmaps/
/api/interviews/
/api/dashboard/
```

Use DRF serializers and permissions.

71. GIT WORKFLOW

Use Git from the beginning.

Suggested commits:

Initial Django project

Configure MySQL

Add authentication

Add career profile

Add resume upload

Add Job Description management

Add parsing service

Add skill matching

Add scoring engine

Add Gemini integration

Add roadmap generation

Add interview preparation

Add dashboard

Add tests

Prepare deployment

Do not wait until the end to initialize Git.

72. TECHNOLOGY CONSTRAINTS

MUST use:

Python

Django

Django REST Framework

MySQL

HTML

CSS

Vanilla JavaScript

Bootstrap 5

Chart.js

Google Gemini API

Use:

pdfplumber

python-docx

for document extraction.

Optional:

python-dotenv

djangorestframework-simplejwt

Do NOT introduce:

React

Next.js

Node.js backend

MongoDB

FastAPI

Flask

Kubernetes

Kafka

RabbitMQ

Elasticsearch

Vector database

Custom ML model

unless a later requirement explicitly changes the architecture.

73. WHAT NOT TO BUILD IN MVP

Do NOT build:

- LinkedIn API integration
- GitHub profile scraping
- automatic job scraping
- automatic job applications
- payment system
- social network
- recruiter portal
- company portal
- real-time chat
- custom machine-learning model
- mobile application
- Chrome extension
- complex recommendation engine
- microservices

These belong in future versions.

74. FUTURE VERSION 2

Possible improvements:

- job board integrations
- GitHub profile analysis
- portfolio analysis
- LinkedIn profile analysis where legally/API-supported
- more detailed resume scoring
- automated job matching
- email alerts
- advanced progress analytics
- resource recommendations

- PDF career reports
-

75. FUTURE VERSION 3

Possible advanced features:

- personalized career paths
- labor-market trend analysis
- salary insights
- company-specific preparation
- skill demand forecasting
- AI mock interviews
- GitHub project quality analysis
- career transition recommendations

These are NOT part of MVP.

76. PRIVACY AND ETHICAL REQUIREMENTS

The application must clearly communicate:

SkillBridge AI provides career guidance based on the information supplied by the user. It does not guarantee employment or selection.

Do not make discriminatory recommendations based on protected characteristics.

Do not infer sensitive personal attributes.

Do not fabricate candidate experience.

Do not encourage candidates to lie on resumes.

77. ACCEPTANCE CRITERIA

The MVP is complete only when:

- User can register
- User can log in
- User can create career profile

- User can upload PDF resume
- User can upload DOCX resume
- Resume text extraction works
- User can paste Job Description
- User can upload Job Description
- Job Description extraction works
- Skills can be extracted
- Skills can be normalized
- Resume and JD skills can be compared
- Matched skills are displayed
- Partial skills are displayed
- Missing skills are displayed
- Required skills are prioritized
- Readiness score is generated
- Score calculation is documented
- AI analysis works
- AI response is validated
- AI failures are handled
- Skill-gap recommendations are generated
- Personalized roadmap is generated
- Roadmap items can be marked complete
- Interview questions are generated
- Resume improvement suggestions are generated
- Analysis history works
- Analysis comparison works
- Dashboard analytics work
- User data is isolated
- Automated tests exist

- .env is excluded from Git
 - README is complete
 - Project runs reliably locally
 - Project is prepared for Render deployment
-

78. DEVELOPMENT PRIORITY

Build in this order.

Phase 1 — Foundation

- Django setup
- MySQL
- authentication
- base UI
- career profile

Phase 2 — Documents

- resume upload
- Job Description input
- PDF extraction
- DOCX extraction

Phase 3 — Analysis Engine

- skill extraction
- normalization
- matching
- scoring
- skill-gap analysis

Phase 4 — AI

- Gemini service
- structured prompts
- structured responses

- response validation
- recommendations

Phase 5 — Career Intelligence

- roadmap
- project recommendations
- interview questions
- resume suggestions

Phase 6 — Tracking

- history
- progress
- re-analysis
- comparison

Phase 7 — Dashboard

- charts
- analytics
- readiness trend
- skill coverage

Phase 8 — Quality

- testing
- security
- error handling
- README
- API documentation

Phase 9 — Deployment

- production settings
- environment variables
- static files
- Unicorn

- Render preparation

79. IMPORTANT PRODUCT PRINCIPLE

SkillBridge AI is NOT:

"Upload resume → get ATS score."

It is:

Resume

+

Job Description

↓

Career Intelligence

↓

Skill Matching

↓

Gap Prioritization

↓

Readiness Score

↓

Learning Roadmap

↓

Practical Projects

↓

Interview Preparation

↓

Progress Tracking

↓

Re-analysis

The product should help the user answer:

"What should I do next to become a stronger candidate for this specific role?"

80. EXPECTED INTERVIEW DISCUSSION

The finished project should allow the developer to confidently explain:

Python

- file processing
- APIs
- data structures
- error handling
- modular design

Django

- models
- ORM
- authentication
- views
- URLs
- middleware
- file uploads

Django REST Framework

- serializers
- API views
- permissions
- JWT
- HTTP status codes

MySQL

- relationships
- normalization
- indexes

- joins
- query optimization

AI

- Gemini API
- prompt design
- structured outputs
- hallucination prevention
- API failure handling
- context limitations

Document Processing

- PDF extraction
- DOCX extraction
- validation
- file security

Recommendation Logic

- skill matching
- scoring
- prioritization
- roadmap generation

Security

- authentication
- authorization
- user data isolation
- file validation
- environment variables

Product Thinking

- why the problem exists
- who the users are

- why the MVP focuses on skill-gap analysis
 - how the product could become a SaaS platform
 - limitations of AI-generated recommendations
-

81. FINAL PRODUCT DEFINITION

At completion, a user should be able to do this:

Create Account



Set Career Goal



Upload Resume



Paste Job Description



Analyze



See Readiness Score



See Matched Skills



See Missing Skills



See Priority Gaps



Receive Personalized Roadmap



Practice Recommended Projects



Prepare Interview Questions



Track Progress



Re-analyze



Measure Improvement

The final product should feel like **Version 1.0 of a real career intelligence SaaS product**, not a resume parser or college CRUD application.

82. FINAL IMPLEMENTATION INSTRUCTION

Using this document as the **single source of truth**, build the SkillBridge AI MVP from the ground up.

Before implementing any feature, verify that it is consistent with this specification.

If an implementation detail is unspecified, choose the simplest maintainable solution that fits:

Python + Django + DRF + MySQL + HTML/CSS/JavaScript + Gemini.

Do not unnecessarily expand the scope.

Do not replace the specified technology stack.

Do not create fake AI results.

Do not fabricate candidate information.

Do not claim that AI predictions guarantee employment.

Every major feature must actually work.

The application must be:

- functional
- secure
- maintainable
- testable
- documented

- responsive
- locally runnable
- ready for later Render deployment

Build the system modularly so future versions can add job integrations, GitHub analysis, portfolio analysis, mock interviews, and advanced career intelligence without requiring a complete rewrite.

Primary success criterion:

A user should be able to provide a real resume and a real Job Description and receive a useful, explainable answer to:

"How prepared am I for this role, what am I missing, and exactly what should I work on next?"